

SEMWEAVER: Provenance-Guided Refinement of LLM-Generated Vulnerability Checkers

ANONYMOUS AUTHOR(S)

LLM-generated vulnerability checkers can miss a vulnerable revision or warn on both sides of its patch. We present SEMWEAVER, an automated refinement workflow that exposes provenance-labelled compiler representations and checker-execution observations, and study how this context changes checker code, warning coverage and report burden. The workflow retains warning recovery or lower fixed-side report counts after normal paired execution; this operational gate does not certify target correctness.

All 39 frozen, generated-only KNighter CSA checkers enter the comparison. With GPT-6-Luna, native SEMWEAVER reaches 23 warning-positive vulnerable revisions versus 15 for KNighter’s actual refinement loop, with patch-version $F_1 = 0.575$ versus 0.448. Native and no-internal refinement have nearly identical F_1 , while native produces more fixed-side reports. In the complete 15-case noisy stratum, native reduces fixed reports from KNighter’s 69 to 44, mainly through one subject. Three paired decodes on 12 subjects show sampling-dependent benefits. An audit of all 11 non-tied comparisons, source-derived perturbations of four mechanisms, and repeated selected-clue interventions distinguish backend-recognition repairs from their remaining semantic obligations. In one checkpoint, target-line recovery repeats, but two of three full-evidence draws report along non-NULL paths. A separate automatic CodeQL example and three-model subset provide bounded feasibility and sensitivity evidence. The results connect useful local repairs, broader matching and invalid diagnoses to concrete code changes; they support neither universal precision gains nor unseen-site generalization.

CCS Concepts: • **Software and its engineering** → **Automated static analysis**; • **Security and privacy** → **Systems security**.

Additional Key Words and Phrases: static analysis, vulnerability checkers, large language models, patch-guided refinement

1 Introduction

Static vulnerability checkers encode conditions that distinguish unsafe code from safe uses of the same operations. Industrial analyzers demonstrate their value [4, 7, 15, 49], but checker authoring requires domain knowledge and familiarity with the analysis framework. Patch-driven synthesis generates a detector from a historical fix. KNighter [59] generates CSA checkers and refines them using compilation, reports and validation on vulnerable and fixed revisions.

An apparently small checker defect can lie between the source-level mechanism and the backend representation: a cleanup attribute names a lowered callback, a location callback supplies a cast-wrapped expression, or a state lookup uses a different region from its update. Patch, source and ordinary reports can support debugging, but need not make the relevant mismatch explicit. Conversely, recognizing the backend representation does not establish the guard, object identity or owning-scope condition needed for a valid warning. This distinction matters when refinement is accepted using warning presence rather than an independently validated diagnosis.

SEMWEAVER adds bounded, provenance-labelled analyzer context to post-synthesis editing. Compiler representations and observations of checker execution supplement the patch, source and ordinary validation feedback. The approach builds on iterative feedback-guided refinement [40, 53]; it does not introduce checker editing or paired validation, which KNighter already performs. Its technical focus is the additional compiler/checker observation interface and explicit

accounting for origin, scope and unavailable evidence. CSA and CodeQL offer different representations [2, 18, 37]; only records with retained backend-produced payloads and bindings count as analyzer-internal evidence.

We evaluate the resulting workflow and the validity of its apparent improvements. All 39 selected generated-only CSA checkers enter KNighter’s actual loop and both SEMWEAVER variants. Native reaches 23 warning-positive vulnerable revisions versus KNighter’s 15, but KNighter’s report-driven policy makes no recovery edit on the 24 initially silent inputs. No-internal already reaches 21. Native and no-internal have nearly tied version-level F_1 , with 123 versus 68 fixed-side reports. The complete preexisting 15-case noisy stratum instead has 44 native fixed reports versus KNighter’s 69, concentrated in G24. These are distinct coverage and burden outcomes, not interchangeable measures of target detection.

We then connect all 11 non-tied comparisons to actual checker edits and scoped source/path evidence. Four frozen source-derived perturbation suites test selected mechanisms; three paired next-decision draws at each of two checkpoints examine selected native clues. These analyses expose both useful repairs and residual obligations: cleanup recognition can activate a wrong-scope state machine, and sink recognition can recover a target-line warning on a non-NULL path. The resulting contribution is an implementation-grounded account of when an observed refinement supports a target claim, rather than a claim that more analyzer context is uniformly better.

Our contributions are:

- A refinement workflow exposing bounded compiler/checker observations with explicit provenance, source binding and missingness, while keeping model edits and programmatic validation distinguishable.
- A complete comparison of 39 checkers using KNighter’s actual loop and both context variants, with repeated subjects, model configurations and a separate backend example.
- An evidence-linked analysis of the complete non-tie subset and selected interventions, separating recognition repairs, supported target obligations and report burden. This analysis provides concrete counterexamples to treating warning recovery or target-line agreement as semantic validation.

The study is patch-local and selected rather than population-representative. Its source/path interpretations, controlled models and additional phases are reported separately; they do not establish target-recall rates, a predictive evidence selector or broad semantic generalization.

2 Background

2.1 Static Vulnerability Checkers

A vulnerability checker encodes a security-relevant condition within a static-analysis framework. Path-sensitive CSA checkers inspect callbacks, symbolic values, regions and state transitions; CodeQL queries operate over extracted program relations [2, 18, 37]. Industrial use demonstrates the value of static rules [4, 7, 15, 49], but the framework does not make every custom checker’s interpretation correct. Its rule must associate an operation with the right object, guards and lifetime. Report burden also affects use [20, 23]; ranking and actionability prediction concern emitted reports [29, 48], whereas refinement changes the code producing them.

2.2 LLM-Based Patch-Driven Checker Generation

Patches provide compact before/after evidence, building on semantic patching, clone-based vulnerability discovery and patch-derived datasets [16, 22, 28, 43]. They are not complete specifications of all safe and unsafe uses. KNighter generates CSA checkers from Linux fixes and already refines them using compiler feedback, diagnostic/report-path context and validation on the original

vulnerable/fixed pair [59]. Its no-report branch provides no report-driven edit. SEMWEAVER adds compiler/checker observations and attempts recovery from silence; it does not claim to introduce post-synthesis editing or paired validation.

2.3 Analyzer-Internal Intermediate Results

Backend intermediates expose how the analyzer represents the program, while checker-execution observations expose how a generated implementation uses that representation. Such facilities build on established graph, data-flow and interprocedural analyses [1, 25, 36, 57]. They can clarify a mismatch without proving the reported vulnerability. We therefore distinguish backend-produced representations and observations, analyzer diagnostics, and source-derived context by actual provenance (Section 4.2). A source window does not become an internal fact because an analyzer-labelled collector emits it; an internal fact does not become a correctness certificate because its raw origin is verified.

3 Motivating Example

This section traces an automatic refinement of the KNighter-derived CSA checker for Linux commit 97cba232549b, which repairs an off-by-one access in the AMD display driver [52, 59]. The example has an explicit safety relation: the largest look-ahead index must be smaller than the extent of the array actually accessed. Both the starting and refined checkers use checkASTCodeBody; the edit adds a missing bound relation to the existing analysis.

The bug and its patch. The member `dc->links` has declared capacity `MAX_PIPES * 2` (Figure 1, ①). We abbreviate this constant as N in the figure; this is a presentation alias, not a new program variable. The function `get_host_router_total_dp_tunnel_bw` iterates over candidate links and, on a matching host-router branch, reads both `links[i]` and `links[i + 1]` (M10–M11, ③). With the original condition $i < N$ (M5, ②), the loop permits $i = N - 1$, so the look-ahead can reach index N , outside the array. The patch changes the bound to $i < N - 1$ (M6, ④), which limits the largest look-ahead index to $N - 1$. Other branch conditions determine whether an access occurs, but the patched loop bound already prevents this final iteration from reaching an out-of-range look-ahead.

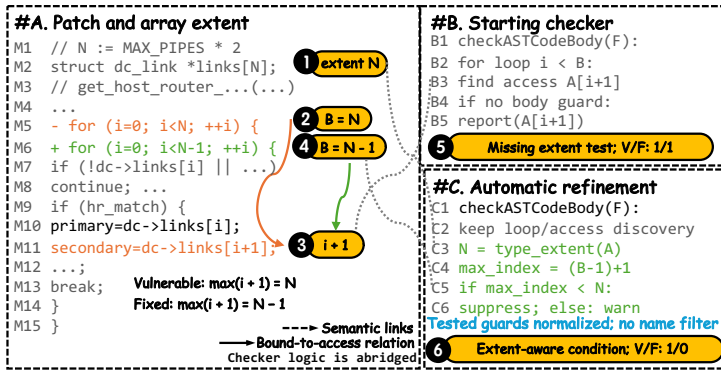


Fig. 1. **Capacity-aware look-ahead refinement.** Abridged code: N denotes `MAX_PIPES * 2`; `hr_match` denotes host-router equality. Original/refined vulnerable/fixed alerts are 1/1 and 1/0 in independent replay. The 14-response, model-produced illustration used fixture feedback and is separate from the one-response main-study checker. Orange/green solid arrows relate vulnerable/fixed bounds to the access; gray dotted links denote semantic correspondences. Code colors mark unsafe/patched bound and capacity logic.

Why the baseline checker fails. The generated checker already recognizes an induction variable, a loop bound, and a look-ahead array subscript. It reports an access when it does not recognize a protective condition in an enclosing `if` (B1–B5, ⑤). That rule overlooks a guard carried by the loop condition itself. The fixed program still contains `links[i + 1]`, so the checker reports on both revisions despite the reduced loop bound. The independent patch-related-object replay yields 1/1 alerts. The refinement problem is therefore to interpret the relation between the loop bound and array extent while retaining the unsafe access.

Evidence-guided refinement. The available bundle contains four provenance-labelled records: two strictly internal Clang 18 CFG/call-graph records and two analyzer-output records. The internal records anchor the affected loop and branch context to raw dumps from `debug.DumpCFG` and `debug.DumpCallGraph`; patch and source context identify the bound change. The declared array extent is read from the AST type by the emitted checker. It is not a path-feasibility proof supplied by the evidence bundle.

The initial GLM-5.3-Flash edit adds a capacity-aware condition while retaining the original traversal. For the depicted loop $i < B$, the maximum look-ahead index is $(B - 1) + 1 = B$. A known extent N justifies suppressing the warning only when this index is smaller than N (C3–C6, ⑥). Consequently, the vulnerable $B = N$ remains reported and the fixed $B = N - 1$ becomes silent. This condition depends on type and index relations rather than the patched function name, a particular array capacity, or adjacency of the relevant statements. The initial edit uses two model calls and passes the independent Linux replay at 1/0.

Checking the example beyond its patch. Additional perturbation probes expose shortcomings in the initial candidate’s fallback and inherited guard recognition. Model-driven quality continuations receive these hash-bound diagnostics and repair the checker; the raw model edits and failed intermediate versions are retained. Four automatic GPT-6-Luna-high quality continuations use 12 additional model calls (14 including the initial Flash refinement). The final frozen checker retains 1/0 on the original Linux pair. It passes all 28 core fixtures, six boundary fixtures, 20 additional guard/bound fixtures, and 60 renamed/capacity-varied executions of those 20 forms: 58/58 unsafe and 56/56 safe executions overall. The labels for the latter 80 executions were also checked with AddressSanitizer [51]. These are finite experimental fixtures, not 114 independent bugs or an unseen-data confirmation set. The fixtures and diagnostic feedback were prepared through the research workflow with coding-assistant support; every scored checker-code edit is model-produced. The tests support robustness to the tested name changes, unrelated statements, constant capacities, and equivalent unit-step-loop/guard forms. Arbitrary aliasing, dynamic extents, and non-unit loop updates remain outside this claim. This illustrative continuation is separate from the frozen 39-case results and does not establish a causal benefit from internal records alone. The one-response main-cohort G21 checker also reports 1/0, but passes only 74/114 of these same fixtures. The stronger 14-response illustration is not substituted into the 39-case scores.

4 Approach

SEMWEAVER is a post-synthesis refinement layer for LLM-generated vulnerability detectors. Given a draft CSA checker or CodeQL query, its motivating patch, source tree and validation scope, it proposes edits using bounded analyzer context and retains operational warning recovery or report reduction. Whether an edit establishes the target semantic condition is evaluated separately.

Figure 2 connects frozen detector/patch/source inputs to rule-guided evidence assembly and a provenance-labelled bundle. The editor updates the latest attempt; paired outcomes update a separate retained detector under Equation 3. Rejected attempts remain available as feedback. Audit artifacts retain source/raw-output bindings, origins, missingness, prompts, replies and all attempts.

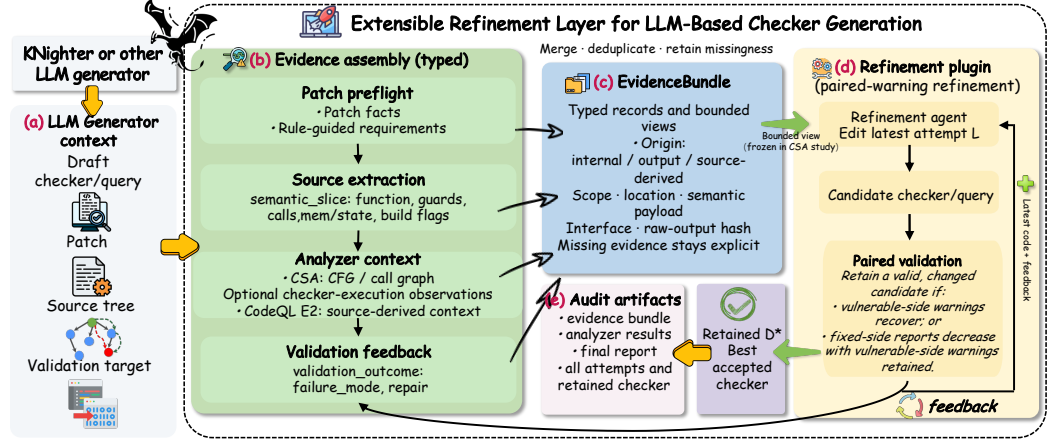


Fig. 2. **SEMWEAVER architecture.** Green groups evidence assembly, blue the provenance-labelled bundle, yellow the editor and paired validation, and purple the best retained checker; colors indicate roles, not correctness. The lower path returns latest code and feedback to the editor; accepted code also enters the audit artifacts. Retention tracks vulnerable-side warning presence and fixed-side burden, not target correctness; fixed silence is secondary. All attempts remain auditable.

These records certify inspectable origin and execution, not the correctness of an LLM explanation or generated checker.

4.1 Evidence Selection and Exposure

The central design choice in evidence assembly (Figure 2b) is *what to collect and expose*. Analyzer outputs can be large and heterogeneous; passing them wholesale to the editor obscures patch-relevant relations. The preflight planner represents candidate requirements by role and priority, while source/function scope and bounded views limit what the editor sees. In the strict CSA comparison, collection is precomputed and the visible view is frozen, so its result does not measure demand-driven routing.

Rule-guided preflight and role requests. The implementation has no separately trained or prompted patch-mechanism classifier. PatchFactsExtractor parses the patch and metadata for affected functions, added guards, risky calls, and type widenings. EvidencePlanner takes a primary pattern from analysis metadata when available or infers it from those facts, then assigns fixed priorities to requirements: patch anchoring is 100, lifetime/state requirements for double-free or use-after-free are 92/89, and a guard for null or bounds cases is 90. It sorts by descending priority with evidence type as a tie-breaker and retains at most eight planned requirements in the default configuration. Table 1 summarizes mechanism-to-role examples; it is not a classifier confusion matrix or a promise that every role is analyzer-internal. The normalizer does not learn or re-rank facts. In the general workflow the editor may request evidence types, with duplicate and request-count limits. For the audited CSA comparison, static records are precomputed and additional requests disabled. The selected v43 implementation resolves compiler references into at most four function-local CFG witness cards, selected with patch/report scope; graph dominance is not path feasibility. It contrasts compiler/checker state associations and helper spelling with the detector, appending available execution observations. No-internal receives neither native witness nor dynamic views,

Algorithm 1 Paired-Warning Evidence-Guided Refinement**Require:** detector D , patch P , source S , scope V , backends A , budget K **Ensure:** refined detector D^* and audit report

```

1:  $R_0 \leftarrow \text{VALIDATE}(D, P, V)$ 
2:  $E_p, E_s \leftarrow \text{PREFLIGHT}(P), \text{EXTRACTSOURCE}(P, S, D)$ 
3:  $E_a \leftarrow \bigcup_{a \in A} \text{COLLECTNATIVE}(a, D, P, S)$     $E_v \leftarrow \text{FEEDBACK}(R_0)$ 
4:  $B \leftarrow \text{NORMALIZEMERGE}(E_p, E_s, E_a, E_v)$ 
5: if  $\text{PDS}(D, P, V)$  then return  $\text{AUDIT}(D, B)$ 
6:  $D^* \leftarrow D$     $L \leftarrow D$     $T \leftarrow \text{INITIALSTOPSTATE}$ 
7: for  $i = 1$  to  $K$  do
8:    $V_i \leftarrow \text{BOUNDEDVIEW}(B)$     $C_i \leftarrow \text{AGENTEDIT}(L, P, V_i)$ 
9:    $R_i \leftarrow \text{VALIDATEIFCOMPILABLE}(C_i, P, V)$ 
10:  if  $\text{ACCEPT}(C_i, D^*)$  then  $D^* \leftarrow C_i$ 
11:   $L \leftarrow C_i$     $E_i \leftarrow \text{FEEDBACK}(R_i)$ 
12:   $B \leftarrow \text{MERGE}(B, E_i)$ 
13:   $T \leftarrow \text{UPDATESTOPSTATE}(T, R_i, D^*)$ 
14:  if  $\text{STOP}(T, K)$  then break
15: end for
16: return  $\text{AUDIT}(D^*, B)$ 

```

► Equation 3

but keeps patch, frozen source and ordinary feedback. This bounded deterministic exposure is not a classifier or a measured optimal routing policy.

Table 1. Rule-guided evidence requirements by patch mechanism (representative); names are persisted record types, not gold labels or measured classifier outputs.

Patch mechanism	Possible record types	Backend
Cleanup edge / double free	allocation_lifecycle, state_transition, path_guard	CSA
Bounds widening / overflow	semantic_slice, dataflow_candidate, state_transition	CSA/CodeQL
Initialization / null deref	state_transition, path_guard	CSA
Use-after-free / escape	allocation_lifecycle, call_chain	CSA/CodeQL
API misuse / missing check	semantic_slice, call_chain	CodeQL

4.2 Evidence Sources and Roles

Four collectors execute in sequence; Figure 2b displays their outputs and Figure 2c records provenance. **Patch preflight** locates changed files/hunks and mechanism context. **Source extraction** resolves function, guard, call, state and build context in bounded views, without repository-scale dumps.

Analyzer evidence is classified by its actual origin, not by the collector’s name. We call a record *analyzer-internal* only if its payload is derived from a backend-produced intermediate representation, identifies the interface and raw output, and is bound to a source revision, file, and function. In the measured CSA cohort the interface is Clang 18’s debug.DumpCFG and debug.DumpCallGraph; the persisted UTF-8 dump is parsed into schema `semweaver.csa_cfg_snapshot.v1`, with a raw-output SHA-256 and source-scope binding. These dumps provide CFG branches and value bindings and call edges. They do *not* themselves prove a path-feasible lifetime or ownership transition. Analyzer diagnostics and alert counts are *analyzer output*, not internal state. Patch diffs, source windows, and compile-database-assisted source extraction are *source-derived* fallbacks even when

a CSA or CodeQL collector emits them. A live CodeQL database query could meet the internal-evidence definition if its query, database revision, and tuples were retained; we do not attribute source-window-derived CodeQL facts to that category.

The selected implementation also observes instrumented CSA callbacks, abstract values/regions and checker-state associations. Raw JSONL uses `semweaver.checker_trace.v2`; bounded feedback uses `semweaver.trace_feedback.v1`. This is not concrete execution or exhaustive symbolic-state proof. Original/instrumented scans must preserve diagnostics on both revisions; caps, live fixtures and input hashes distinguish valid absence from broken instrumentation.

Extraction must handle verbose, backend-specific intermediates and instrumentation that may shadow identifiers, exceed caps or see no eligible callback. Revision, architecture, scope and parity checks retain missing observations as unavailable. Table 2 partitions 193 records across all 39 subjects. Its 81 native records comprise 45 guards, 27 state-transition and 9 allocation-lifecycle records; all 61 source-derived records are patch diffs, not source-window fallbacks. Attached context is not itself native. Dynamic availability is per-attempt (RQ3).

Table 2. Evidence primitives and provenance in the audited 39-case CSA cohort. One record may contain several CFG primitives; counts are by record origin, not primitive.

Origin	What the persisted record contains	Records
Analyzer-internal	Clang CFG guards, value bindings and call edges; raw-output and scope binding	81
Analyzer output	CSA diagnostics and paired validation reports; no internal state	51
Source-derived	Patch-diff records here; other interfaces may emit source-window fallbacks	61

Validation feedback records compilation, execution health and paired warnings: symptoms to diagnose, not semantic ground truth.

The seven persisted types are `patch_fact` (changed statements), `semantic_slice` (bounded statements, guards, API terms and symbols), `dataflow_candidate` (possible propagation), `call_chain` (call context), `path_guard` (control), `allocation_lifecycle` (resources), and `state_transition` (abstract state). “Barrier,” “type filter,” and “API usage” interpret record content; they are neither extra types nor proven path feasibility.

When a selected role yields no evidence, the bundle retains an explicit missing-evidence marker rather than treating the absence silently, so the refinement agent knows what was requested but unavailable.

4.3 Evidence Bundle Schema

The normalizer merges all collected facts into the EvidenceBundle shown in Figure 2c. Each fact becomes an EvidenceRecord carrying a type, the contributing analyzer, scope, source location, payload, provenance, and an optional EvidenceSlice (the compact semantic content consumed by the agent). The schema separates program facts from analysis rules in the spirit of relational analyzer frameworks [2, 5, 24], but uses a smaller role vocabulary tailored to checker refinement. Formally:

$$\begin{aligned}
 B &= \text{Norm}(E_p \cup E_s \cup E_a \cup E_v), \\
 E_a &= E_{\text{CSA}} \cup E_{\text{CodeQL}}, \\
 r \in B.\text{records} &= \langle \text{type}, \text{analyzer}, \text{scope}, \\
 &\quad \text{location}, \text{payload}, \text{provenance}, \text{slice} \rangle.
 \end{aligned} \tag{1}$$

Here E_p contains patch facts, E_s bounded source slices, E_a analyzer-internal representations or outputs filtered by selected roles (Section 4.1), and E_v validation outcomes. Each record’s provenance has one of three origins: `analyzer_internal`, `analyzer_output`, or `source_derived`. The normalizer deduplicates records and marks missing evidence; origin is neither a confidence score nor a proof of semantic correctness.

The type-to-backend mapping reflects available interfaces: CSA CFG/call-graph dumps inform guard, state, and call-chain records; CodeQL queries can support flow and relation records when a live database is available. These are record types, not seven distinct backend analyses. Cross-function call edges provide context, but the present CSA dump does not establish an interprocedural path or ownership proof. Provenance remains visible when a type is populated only by a fallback.

The bundle is bounded by design: scope and selected views limit raw dumps. The matched pipeline contrast and repeats remain subject to selection history, stochastic draws and bundled native components; they do not isolate each representation’s causal effect.

4.4 Paired-Warning Refinement

The editor in Figure 2d receives the current detector, patch, baseline validation notes and a bounded evidence view. The audited CSA condition freezes the visible view before editing and disables additional role requests, measuring preselected context rather than retrieval policy.

Tool-mediated editing uses attached evidence, detector edits and validation commands [40, 50, 53, 61]. Where enabled, requests replay attached records rather than explore the repository; caps, unused roles and missing evidence are logged. The matched condition makes zero requests.

Model/programmatic boundary. LLMs may produce patch-analysis metadata, checker edits and enabled role requests. Preflight rules, raw-output hashing/scope binding, normalization, compilation, paired replay and acceptance are programmatic. The LLM neither produces backend dumps nor assigns origin or PDS. The audit retains exact starting checkers, prompts/replies, visible roles, raw evidence, candidates and decisions; model explanations do not certify analyzer origin. Editor prompts provide current/starting checker and patch paths, bounded source/evidence, baseline V/F symptoms, backend identity and edit/validation rules. They require in-place mechanism-based edits rather than patch-spelling matches, preservation of vulnerable triggers, and compilation before paired acceptance. Repair prompts supply actual compiler diagnostics. The artifact indexes available English templates and rendered exchanges; missing historical raw exchanges are marked, never reconstructed.

Retention gate. Changed candidates must compile and receive normal paired execution. Equation 3 retains warning recovery from silence, or lower fixed reports with a nonzero vulnerable signal. PDS is stronger but not mandatory. The native-only controlled-fixture guard preserves required existing hits; it is not a target oracle. Silencing an existing vulnerable signal is rejected. A comparative disadvantage against control may instead reflect failure to recover from initial silence or a smaller improvement.

Feedback loop. The next reply edits the latest attempt using build/paired feedback; the best retained detector is separate. Rejected/invalid edits remain visible. Algorithm 1 and the evaluation protocol specify stopping. Interruptions remain unscored and are repaired without resampling consumed prefixes. The noisy-15 view reuses full-cohort outputs, not a separate per-case-budget experiment.

Backend extension surface. An adapter must provide registration, evidence/provenance mapping, dependencies, execution/result decoding, and paired validation. Shared modules still contain backend branches. Four enumerated CodeQL-specific modules occupy 2,438 physical lines, including comments and generation code: an existing footprint, not code to rewrite for each backend.

With a usable analyzer and patch pair, we estimate that a familiar developer using Codex can complete basic adaptation within an afternoon by reusing these interfaces. This is a planning estimate, not measured labor or a scalability result. New native-evidence semantics and broad validation require separate work. The artifact lists prerequisites and tasks. A new bug mechanism needs a role mapping and safe/unsafe fixtures, and a new extractor only for unavailable relations.

5 Evaluation

We evaluate automated patch-related warning recovery and report reduction, not whole-kernel bug discovery. Checker outcomes and execution health are recorded separately.

5.1 Research Questions

RQ1: Generated CSA checkers. From identical starting artifacts, how do KNighter’s actual loop and SEMWEAVER affect vulnerable-version warning coverage, fixed-side report burden and paired discrimination?

RQ2: Cross-backend feasibility and cost. Can an archived automatic CodeQL query be refined and independently replayed, and what integration/execution costs are observable?

RQ3: Evidence sensitivity. How do native/no-internal outcomes differ across the cohort and repeated subset, and what do the resulting edits and targeted follow-up establish about their reported gains?

RQ4: Model configuration sensitivity. How do three editor configurations perform on those 12 subjects under common reply/output ceilings?

5.2 Subjects

E1 freezes one generated CSA checker per selected Linux commit from KNighter’s artifacts. Of 286 records on 60 commits, 241 on 39 commits lack an upstream refinement-status row. Per commit, the fixed input rule ranks by upstream TP, then TN, then lowest checker index, taking nonempty repaired source. This non-random ranking selects inputs; independent paired replay supplies outcomes. Failed/noisy refinements do not remove subjects. Initially 15 checkers warn on both revisions and 24 are silent on both; zero PDS is the starting state, not KNighter’s refinement result. No manual completion enters scored portfolios.

All 39 exercise actual KNighter refinement and native/no-internal SEMWEAVER. The noisy 15 are the complete preexisting report-refinement stratum, not a result-selected subset. For the other 24, KNighter’s no-report branch compiles without model edits, whereas SEMWEAVER attempts recovery. This system comparison does not imply KNighter attempted and failed 24 recovery edits.

The selected whole implementation is v43, chosen after comparing two configurations on six experimental subjects. Exact compatible executions on those subjects are reused in E1; selection probes are not an independent confirmation set. Source revisions, reuse equivalence and earlier trials remain available. G04 uses the architecture-correct ARM64 scope and evidence, not a zero substituted for earlier wrong-object execution.

Figure 3 summarizes E1–E4. E2 uses an automatic CodeQL input from an archived independent generator; historical mixed/manual 20-row claims are withdrawn. CodeQL is neither pooled with CSA nor compared against KNighter. E3 covers all 39 main pairs plus the repeated subset; E4 is native-only, one decode per model.

The 12-subject selection uses starting status, not refinement outcomes. Within the original 14 warning-positive/fixed-noisy and 25 vulnerable-silent cases, we sort SHA-256 hashes of a fixed, artifact-recorded salt plus case ID and take the first 4 and 8, respectively. Later scan repair corrected G08 from 0/0 to 2/2: the unchanged subset becomes 5 noisy and 7 silent, with no replacement. Membership predates this final round; selection is not stratified by vulnerability class and includes

no buffer-overflow/out-of-bound cases. Repeated decodes are not independent bugs; E4 is neither full-39 nor multi-run cross-model robustness.

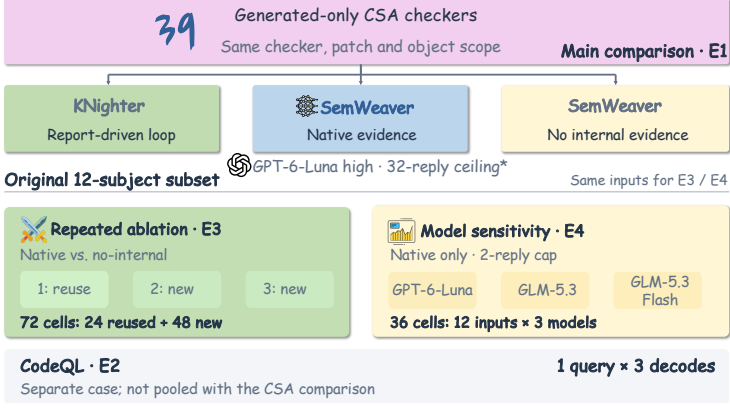


Fig. 3. **Experimental design and reuse.** E3’s first decode reuses E1; E4 uses the same 12 inputs, not another cohort. Cells are not independent bugs. *Audited natural-stop reuses retain original caps; shared ceilings do not imply equal realized effort.

5.3 Metrics

Primary outcomes are vulnerable-version warning presence and fixed-side report burden. Let $n_v(D)$ and $n_f(D)$ count warnings on the same vulnerable/fixed patch-related object scope. The stronger secondary outcome PDS is:

$$\begin{aligned} H_v(D, P, V) &= \mathbb{1}[\text{alerts}(D, V_{\text{vuln}}, P) > 0], \\ S_f(D, P, V) &= \mathbb{1}[\text{alerts}(D, V_{\text{fix}}, P) = 0], \\ \text{PDS}(D, P, V) &= H_v(D, P, V) \wedge S_f(D, P, V). \end{aligned} \quad (2)$$

PDS is not required for a useful edit. *Warning recovery* makes an initially silent vulnerable revision warning-positive, possibly with fixed noise; *report reduction* lowers fixed reports while preserving vulnerable warning presence. Silencing a previously positive vulnerable revision is rejected. *Comparative disadvantage* means another arm retains a better result, not necessarily regression from the starting checker.

For retained detector D^* and normally paired-executed candidate C , the improvement rule is:

$$\begin{aligned} \text{Accept}(C, D^*) &= \text{AnalyzerOK}(C) \wedge H_v(C) \\ &\wedge (\neg H_v(D^*) \vee n_f(C) < n_f(D^*)) \wedge Q(C). \end{aligned} \quad (3)$$

The editor’s local gate requires a changed artifact before paired assessment. Q preserves required hits on controlled array fixtures in native; it is identically true for no-internal. This native-only guard records zero rejections in E1/E3, but its overhead and policy asymmetry remain disclosed. It is not a target-location oracle.

We binarize each checker/revision by warning presence. Each vulnerable revision is one positive and its fixed revision one negative; precision, recall and $F_1 = 2TP/(2TP + FP + FN)$ describe 78 patch-version decisions, not individual bug sites. Fixed reports measure inspection burden, not confirmed false-positive bugs. G08 warns on allocation failure rather than the registration-failure

leak repaired by its patch; the follow-up places G24’s remaining warnings outside the target life-cycle (Section 6.5). No exhaustive target-recall claim follows. Failed scans are unscored, never substituted as zero warnings.

5.4 Experimental Environment

CSA uses pinned Clang/LLVM 18.1.3 and Python 3.12.3; CodeQL uses 2.26.4, codeql/cpp-all 8.0.3 and no-build databases. Each Linux subject uses its patch commit and parent with allyesconfig, architecture-correct objects and identical starting/baseline/candidate scope. Manifests bind code, evidence and raw paired outputs. Requested model labels and high effort are retained; service labels do not establish immutable revisions. Configured temperature 0 is not enforced sampling control: both Responses adapters omit it. We claim no fixed seed or deterministic decoding.

Generative AI in research. LLMs generated starting checkers and every scored checker edit. Extraction, provenance checks, compilation, paired scans and numerical aggregation are programmatic. Codex assisted with drivers, source-contract analysis, perturbation construction and secondary diagnostic interpretation. The primary follow-up fixtures used patch and source inputs, without detector code or outputs, and were frozen before scanning. G19/G25 semantic reviewers received condition-masked checker/source/PLIST packets, not arm mappings or desired outcomes. Their source/path judgments are AI-assisted, not independent human ground truth; mechanical coverage and hash checks do not validate semantics. Original messages, code, labels, scripts, exceptions and uncertainty are retained for inspection. This research use is separate from the writing-use statement.

5.5 Baselines and Variants

E1 native, no-internal and KNighter share starting checkers, requested GPT-6-Luna high and a 32-reply ceiling. KNighter retains its five-report/two-outer-attempt/report-driven algorithm; its actual emitted artifacts receive independent scans. Its 24 no-report branches consume no reply. Two natural one-response No-FP stops are equivalence-audited reuses with their original eight-reply ceiling retained, not relabelled as fresh32-reply runs; termination occurs before either ceiling. The 15-case view uses these same outputs, not a favorable per-case allocation. Equal maximum replies do not mean equal actual calls, tokens or CPU work.

Within SEMWEAVER, no-internal withholds native compiler and checker-execution views while keeping checker, patch, frozen source context, build/paired feedback and stopping rule. Native receives bounded CFG witness/mismatch views and available dynamics; absence is explicit. E3 repeats the original 12 at 32 replies and 16,384 output tokens per call. E4 evaluates GPT-6-Luna high, GLM-5.3 high and GLM-5.3-Flash high with 2 replies and 65,536 output tokens per call. Actual GLM effort fields are retained. E4 is limited-budget configuration sensitivity, not equal-compute model ranking or cross-model native/control causality. Earlier default-effort, smaller-output and outside-subset histories stay separate.

5.6 Protocol

Each permitted reply is an edit attempt; compilable changed candidates receive independent paired replay. The next call starts from the latest attempted artifact and its failure/paired feedback, while the best retained detector is separate. Normal candidates follow Equation 3. Stopping occurs at positive/fixed-silent saturation, three valid validations without improvement, three consecutive invalid model attempts, or the reply cap. Rejected edits, compile/format failures, observer unavailability, provider interruptions and invalid scans remain distinct.

Repairs preserve original failures and consumed prefixes. Thinking-only received replies are charged, not resent free. One GLM reply contained an explicit action and three edits but omitted

explanatory summary; network-disabled replay supplied only an empty summary, reproduced exact model-specified code and made zero new calls. Healthy zero-callback probes use static evidence without invented traces. Repairs do not authorize manual scored-code edits, changed budgets or failed scans counted as negatives.

5.7 Target and Mechanism Follow-up

We audit all 11 non-tied native/no-internal comparisons, not a favorable subset: 362 condition-expanded retained HTML occurrences and seven separately counted log-only diagnostics. For each report, the audit distinguishes target association, bounded source/API support, and support from the emitted path. Unknown off-target validity remains unknown. All 22 initial-to-selected code diffs are retained. The 28 tied/other cases are not target-audited; no full-cohort target precision or recall is inferred.

For G12, G19, G24 and G25, a separate construction pass receives patch and source inputs without checker code or outcomes and freezes six vulnerable/fixed variants per mechanism before scanning the retained detectors: 48 files and 96 unique scans. G12 models access annotations, not concurrent execution; G24 models logical retirement, not physical kernel UAF. G19’s initial wrap-per adaptation differs from canonical lowering, so four later canonical/renamed controls remain a separate supplement. Four benign cleanup controls are frozen before the subsequent intervention replies. Neither supplement replaces the primary misses; variants are not independent bugs.

At selected pre-edit checkpoints, we replace four native-observation lines for G19 and five for G25 with missing markers, keeping other context unchanged. Both arms retain other native context; G25 already contains an earlier binding repair. Three fresh next decisions per arm use GLM-5.3-Flash, high effort and a 65,536-output-token ceiling: 12 received decisions, not full-loop re-runs or historical GPT attribution. Nine conform to the strict response schema. A separate post-response bounded adapter replay yields 12 applicable edits from the same replies, handling two omitted empty evidence lists and one complete text-carried JSON decision. All three exceptions are withheld outputs. G25 has no scorable strict detector pair and one adapted pair before compiler completion; no edit is rewritten or response resampled.

Two applicable G25 withheld drafts fail compilation. A separately frozen compiler-only supplement uses a common eligibility rule and two-reply cap, stops at first compilation success, and supplies no fixture feedback. Both compile after three additional replies, including one invalid reply with no applied edit; only withheld consumes this extra effort. Applied fixes are local API/type corrections, without assumed semantic equivalence to unbuildable code. Original failures remain unscored; 192 terminal-artifact scans form a separate phase, not new independent draws. Mechanical scores retain final-file/line hits and negative report burden. Condition-masked source/checker/full-PLIST review adds secondary interpretations without changing primary scores.

6 Results

6.1 RQ1: Generated CSA Checkers

Table 3 reports all 39 subjects and the preexisting 15-case noisy stratum. Zero starting PDS is an entry condition, not KNighter’s refinement result. Every portfolio retains the 15 initially warning-positive vulnerable revisions. Among the 24 initially silent subjects, native recovers 8, no-internal 6, and KNighter 0 under its actual no-report policy. These are warning signals, not verified target-recall counts.

Warning coverage and report burden move in different directions: native’s 123 fixed reports exceed no-internal’s 68 and KNighter’s 69; its precision is 0.561 versus 0.618 for no-internal. The

Table 3. CSA comparison: (a) all 39 subjects; (b) the complete preexisting 15-case fixed-noisy stratum, reusing the same outputs. TP/FP/FN are version-warning decisions; fixed reports measure burden and replies count received responses. Color shades 1 – F_1 at .25/.50/.75, not significance.

(a) Full cohort: 39 subjects

Condition	Version outcomes				F_1	Burden / effort	
	PDS	TP	FP	FN		Fixed reports	Replies
Unmodified	0	15	15	24	.435	73	0
KNighter loop	2	15	13	24	.448	69	179
SEMWEAVER, native	5	23	18	16	.5750	123	150
SEMWEAVER, no internal	8	21	13	18	.5753	68	133

(b) Noisy 15: all methods retain TP=15, PDS=2 and $F_1 = .6977$

	KNighter loop	Native	No internal
Fixed reports	69	44	68

nearly tied F_1 summarizes different coverage/noise portfolios and does not establish native superiority. G12 contributes 73 distinct fixed-side warning locations, not duplicates; its broader matching yields 74 vulnerable and 73 fixed reports. This unfavorable tradeoff remains in the aggregate.

In the complete 15-case fixed-noisy stratum (Table 3b), native reduces reports by 25 (36.2%) against KNighter and 24 (35.3%) against control. The effect is concentrated in G24’s $27 \rightarrow 2$ fixed reports; other small differences offset one another. Identical binary fixed positivity and F_1 thus coexist with different report burdens. The follow-up finds G24’s retained reports outside the target lifecycle (Section 6.5); the lower count is therefore not evidence of target-preserving suppression.

6.2 RQ2: Cross-Backend Feasibility

A separate archived automatic ImageMagick CodeQL query starts at 0/0 vulnerable/fixed rows. Table 4 includes two 8/0 recoveries and the third decode’s 22/22 outcome. The successful edit corrects relative-path matching while keeping exact file/function/local-variable filters. This demonstrates patch-local operation on a second format, not broad backend transfer or native-evidence causality. A second query already starts at 2/0 and is not counted as a gain. KNighter is not a CodeQL baseline.

Table 4. Three automatic CodeQL decodes of one query. Counts are vulnerable/fixed output rows; time is recorded agent-round duration, excluding separate paired replay.

Decode	V/F rows	Replies	Tokens	Agent time (s)
1	8/0	1	15,122	87.87
2	8/0	1	14,345	42.74
3	22/22	8	110,282	516.77

Second-backend costs. Table 4 reports the original model runs. Three warm paired replays per query preserve 0/0 and 8/0. Median query-plus-decode time is 12.57 s initially and 36.51 s after refinement with two threads; peak process RSS is 976/1,078 MiB. Database-creation log intervals span 31/30 s, excluding unlogged startup. Developer labor is unmeasured. These one-case existing-environment observations are neither a speedup, cold-install benchmark nor whole-project scalability estimate.

6.3 RQ3: Internal-Evidence Sensitivity

Table 2 reports initial record provenance; Section 4.2 defines native types and source-derived context. Dynamic availability has a separate denominator: 103 of 150 unique native-chain attempts have usable hash-bound observations; 11 are unavailable due to candidate compilation, 18 unsupported context, 13 zero scoped callbacks, 1 reproduced candidate crash and 4 capped optional observations. None establishes safety from absence.

Across 39 subjects, native has six comparative benefits, five disadvantages and 28 ties/other outcomes. The associated edits expose concrete differences in checker behavior: cleanup-wrapper recognition in G19, nested dereference/binding recovery in G25, and null/region/frame handling in G24. G19 adds `__free_kfree` to a cleanup predicate, yielding 15/0 versus control’s 0/0; 15 reports are not 15 bugs. Conversely, control recovers symbolic-region handling in G23 and persistent-field release recognition in G38 without native records. G12 drops CPU-role restrictions, explaining its broad matching and noise. These code observations distinguish repairs from broader matching; a record’s presence or an LLM explanation alone does not prove edit causality.

Subject	Decode 1 (E1)			Decode 2			Decode 3		
G04	=	0/0	: 0/0	=	0/0	: 0/0	=	0/0	: 0/0
G08	=	1/1	: 1/1	—	2/2	: 1/1	=	1/1	: 1/1
G09	+	1/1	: 0/0	=	1/1	: 1/1	=	1/1	: 1/1
G13	=	0/0	: 0/0	=	0/0	: 0/0	=	0/0	: 0/0
G14	—	2/2	: 2/1	=	2/1	: 2/1	=	2/1	: 2/1
G15	=	9/9	: 9/9	=	9/9	: 9/9	=	9/9	: 9/9
G19	+	15/0	: 0/0	+	15/0	: 0/0	=	15/0	: 15/0
G24	+	2/2	: 27/27	=	3/3	: 3/3	=	27/27	: 27/27
G25	+	2/2	: 0/0	=	0/0	: 0/0	=	0/0	: 0/0
G36	=	1/1	: 1/1	=	1/1	: 1/1	=	1/1	: 1/1
G37	=	0/0	: 0/0	=	0/0	: 0/0	=	0/0	: 0/0
G38	—	0/0	: 1/0	=	1/0	: 1/0	=	1/0	: 1/0

Native V/F : no-internal V/F. + benefit; — disadvantage; = tie/other.

Recovery: native 4 / control 1; report reduction: native 1 / control 2; ties/other 28.

Fig. 4. E3: all 36 paired decodes on 12 subjects; cells compare native and no-internal V/F counts. Decode 1 reuses E1; the remaining decodes add 48 condition cells. Color and sign encode the comparative direction.

Figure 4 shows five subjects tying in all three decodes and seven with mixed directions; none benefits comparatively in all three. G19’s native 15/0 recurs, but control catches up in its third decode. G24’s main reduction does not repeat. G14’s $3/3 \rightarrow 2/2$ improves its own start but trails control’s 2/1; later decodes tie. In the main comparison, G05/G23/G38 retain initially 0/0 in native while control recovers warnings, so their disadvantage is not loss of an initial signal. Repeated subjects, selection/reuse and native-only gating limit stronger attribution. Realized call differences under a shared stopping rule are outcomes/mediators, not independently assigned confounds.

6.4 RQ4: Model Configuration Sensitivity

Table 5 uses the original 12 subjects, native only, one decode per model, 2-reply and 65,536-output ceilings with requested high effort. All configurations retain the five initial vulnerable warning signals and recover additional versions. Flash has the strongest observed outcome here, but one decode on this subset does not establish general ranking or variance-controlled model robustness.

Received replies total 24/23/21 respectively. Capped, format-invalid and compile-failed replies remain charged; a healthy previous detector is retained when no valid improvement occurs. These

Table 5. Limited-budget native sensitivity: 12 subjects per model, not full-39 robustness. Computation differs despite common reply/output ceilings. F_1 shading follows Table 3.

Editor	TP	FP	PDS	F_1	Fixed reports	Replies
Unmodified	5	5	0	.455	42	0
GPT-6-Luna high	7	6	1	.560	40	24
GLM-5.3 high	7	5	2	.583	16	23
GLM-5.3-Flash high	9	5	4	.692	11	21

are complete workflow outcomes, not successful code generation on every call. No no-internal arm is run across models, so E4 cannot isolate cross-model evidence effects.

6.5 What the Observed Improvements Establish

Complete non-tie audit. Of the six warning-based native benefits, G12/G19/G25 have bounded source-supported target reports; G09 has conditional support; G24/G35 have no supported or conditional retained target report. Of the five control advantages, G01/G23/G38 have bounded source-supported target reports and G05/G14 have conditional support. These describe the complete selected subset, not corrected success rates or independently adjudicated target accuracy.

The code changes explain why these categories differ. Both G01 arms recover the governing loop, but only control adds a capacity/break guard; native’s fixed reports contradict the actual guard. G19 adds a lowered cleanup name, while G23 control recovers symbolic-region and nested-member handling. G25’s native changes add nested sink recognition and destination binding. Recognition therefore improves in both arms, but is separate from guard and state correctness. G05 preserves uncertain initialization; G09 broadens cleanup handling; G12 removes actor restrictions; G35 broadens status/condition reporting. Their recovered reports range from conditional target concerns to unrelated, contradicted reports. G38 control preserves field identity while relaxing a usage gate, with bounded reference-lifetime support.

Report reduction can also change the surviving report population. G14’s arms project onto different completion/free events; one fixed control report concerns a different supported stack-lifetime risk. G24 removes 25 reports per side, but its two retained native reports concern destroyed owners rather than the intended surviving-owner lifecycle. This does not prove loss of a previously valid witness; it does show that nonzero vulnerable warning presence cannot certify target preservation.

Frozen perturbations. Each primary mechanism has six positive and six negative fixtures. G12 reports at a target location in all positive fixtures and none of the negative ones, within its annotation model; its remote-write note still points to a local reset. G19 is silent on its adapted-wrapper suite. Its separate canonical control yields one supported early-exit report plus a wrong-scope allocator-return report, and misses the renamed wrapper. G25 has target-location reports in all six positive and five negative fixtures. Starting/control artifacts are silent for these three cases. In G24, starting/control has target-location reports in all six positive and six negative fixtures with 108/108 total reports; native has none with 18/12 total reports, although secondary review supports guard warnings outside the frozen target-line list. These disagreements remain visible rather than changing the scoring rule after inspection.

Selected-clue intervention. All three full G25 decisions peel outer sink-expression casts; no withheld decision does. Two full decisions also change bookkeeping. Table 6 reports the separate compiler-completed artifacts: full recovers target-line presence with negative-side noise, while withheld remains silent. This rules out uncompiled controls as the sole reason for terminal silence, but does not establish three valid detection gains: all 34 reports from full repetitions 2/3

take $\text{argc} \leq 1$, which selects a concrete non-NULL object. Full repetition 1 has six outer count-access reports admitting a source-feasible NULL completion, without an explicit NULL-return event, alongside unsupported reports.

Table 6. GLM-5.3-Flash bounded-adapter terminal artifacts after separate compiler-only completion. Each cell is full/withheld. Primary suites have six positive and six negative fixtures; G19 lowering has two of each and benign has four negatives. Hits/alerts are final-location presence, not validated diagnoses. Original strict/adapter failures remain separate (Section 5.7).

Draw	G19			G25		
	Primary positive hits	Lowering positive hits	Benign alerts	Positive hits	Negative alerts	Negative reports
1	0/6	1/2	0/2	6/0	5/0	28/0
2	0/6	1/2	0/2	6/0	5/0	10/0
3	6/0	2/0	1/0	6/0	6/0	6/0

G19 instead exposes a coverage/noise tradeoff. Full repetitions 1/2 retain name-family tests; withheld generalizes registration, while repetition 3 reverses primary coverage. The broader matchers can warn on benign cleanup. Across the reviewed G19 artifacts, supported owning-scope exits coexist with reports at callee returns and a write-only cleanup mistaken for a read. This is selected-checkpoint evidence, not a population effect or a rule predicting useful native context.

Resulting engineering distinction. The same source line can contain a safe pointer-field read and the actual optional-object dereference, while a relevant guard report can lie outside a frozen sink-line list. Thus neither report presence nor final-line association substitutes for the reported object, operation, path and owning scope. The contribution is the evidence-linked explanation of these divergences; the source judgments remain bounded and do not turn the workflow into a semantic verifier.

7 Discussion

What analyzer context contributes. Compiler/checker observations can make a representation mismatch explicit: which cleanup callback is registered, what expression reaches a callback, or which region a state operation references. The G25 intervention links selected wrapper clues with a repeated repair choice. This is useful debugging evidence, but not proof that source inspection could not derive the edit or that its resulting warnings are correct. The control repairs in G01/G23/G38 also establish that representation repair is not exclusive to the native condition.

Activation is not semantic validation. Correcting recognition exposes the rest of the checker. G19’s recovered cleanup tracking still confuses caller and callee scope; G25’s recovered sinks can report on a non-NULL path. Predicate relaxation has similarly different consequences across G05/G09/G12/G38. The observed edit, its target/source obligation and its warning burden should therefore be reported separately. This recommendation follows from inspectable counterexamples, not an evaluated automatic acceptance policy or measured developer-time saving.

How to interpret the comparison. The 39-case results measure operational recovery and burden under the recorded policies. KNighter’s silent-input behavior differs from SEMWEAVER’s recovery objective; the no-internal comparison is more informative about the additional context. Native’s near-tied F_1 , greater full-cohort noise and sampling-sensitive benefits remain central results. G24’s concentrated count reduction is not established target-preserving suppression. These findings qualify useful local repairs rather than imply either universal superiority or that analyzer evidence is useless.

Transfer boundary. The motivating checker supports its finite, fixture-guided transformations, not unseen-family recall. The automatic CodeQL case demonstrates a second detector format, not broad semantic portability or CodeQL-native evidence coverage. Recorded warm execution costs and implementation footprint remain separate from estimated adaptation labor and whole-kernel scalability. None of these boundaries is resolved by the additional source/path audit.

8 Threats to Validity

Internal. Build, instrumentation and scoring failures can distort outcomes. We keep received format/code failures separate from invalid executions, retain consumed prefixes and unavailable observations, and bind raw output to code and scope. All scored checker edits are automatic; old manual/mixed rows remain excluded. Configuration selection overlaps the main cohort, native components are bundled, and the native-only fixture guard, despite zero E1/E3 rejections, remains a policy asymmetry. The checkpoint intervention controls only selected clues in the recorded prompt; it does not isolate all native components or explain the historical full workflow. Its post-response adapter interpretation and compiler-only continuations stay separate from strict first decisions. Equal reply ceilings do not ensure equal realized compute; service aliases and configured temperature do not establish immutable weights or deterministic sampling.

Construct. Version-warning F_1 describes 78 revision decisions, not target precision or recall. Retaining some vulnerable warning does not retain every target report, and fixed reports can concern unrelated properties. Final-line matching also misses object and path distinctions. The follow-up separates primary counts from AI-assisted source/path judgments; source completion is not an exported analyzer-state or runtime witness. G19/G25 reviewers use different mixed-observation conventions, so those labels are not pooled as accuracy. Positive source interpretations remain uncalibrated by independent human adjudication. Some historical zero-scan records lack complete stdout; absence of a retained warning never establishes safety.

External. The 39 representatives come from one generator’s selected Linux commits, not a random sample of all 286 records or independent projects. The complete noisy stratum and silent starts pose different tasks; KNighter’s no-report policy does not attempt recovery. The 11 non-ties, four perturbation mechanisms and two checkpoints are purposive, nested scopes, not new representative cohorts. G12/G24 fixtures model annotation/retirement properties, not executed races or kernel UAF. The motivating fixtures informed its edits. E4 omits bounds cases and uses one decode per configuration; CodeQL remains a single patch-local example with inherited filters.

Conclusion. Neither repeated decodes, reused scans nor transformed fixtures are independent bugs. Native/control F_1 is nearly tied and no E3 subject benefits comparatively in all three decodes. The follow-up establishes inspectable local mechanisms and counterexamples, not their population prevalence, a predictive evidence-selection rule, universal precision improvement or unseen-site generalization. These limits remain even where implementation repairs or source-supported warnings are observed.

9 Related Work

Static Analysis and Checker Languages. Path-sensitive analyzers, abstract interpreters and symbolic engines supply guards, states, paths and diagnostics [3, 6, 13, 15], including industrial deployments [4, 7, 49]. Relational frameworks express rules over program facts [2, 5, 24]; semantic patching encodes systems-code transformations [43]. SEMWEAVER adds no analyzer or query language. Its edited artifact is a generated checker, whose use of those facilities is part of the object being evaluated.

Vulnerability Detection and Patch Evidence. Detection uses taint/data-flow reasoning [1, 25, 36], graph/AST representations [57, 58], and learned token-, slice- and line-level models [17, 34, 35,

47]; dataset and evaluation studies expose semantic robustness concerns [8, 16]. Clone- and patch-driven systems use historical fixes as evidence [22, 28, 33]. Here a patch scopes the refinement task, but neither a changed line nor a warning on that line certifies the intended target property.

LLM-Based Reasoning and Checker Refinement. Code models and repository agents synthesize and analyze programs [11, 19]; LLM DFA decomposes data-flow reasoning and synthesizes tools [56]. Security evaluations show why generated artifacts require validation [26, 44, 45]. KNighter [59] is our nearest workflow comparator: it already generates, compiles, edits and revalidates checkers using patch and report context. Our distinction is the bounded compiler/checker observation supplied to the editor and the recovery objective on silent starts, not the existence or temporal placement of refinement. The no-internal variant and selected-clue interventions examine that additional context at different scopes.

Feedback and Validation. Repair and synthesis use compiler, test or verifier feedback [12, 27, 30, 38, 39, 41, 42, 55]; LLM workflows add iterative feedback, tools and search [40, 50, 53, 60, 61]. Weak validation proxies and repair overfitting are established problems [46, 54]. We do not claim their discovery. Our empirical evidence concerns the predicates and diagnostic obligations of edited static checkers: a representation repair can change report production without establishing the reported object/path/scope property, including on evaluated inputs.

Analyzer Context. LLift uses analyzer reports and post-constraints [31]; Chapman et al. interleave analyzer intermediates and prompting for error specifications [9]; IRIS infers taint specifications with CodeQL [32]. LLM4PFA performs report-feasibility reasoning [14], and ZeroFalse enriches reports before adjudication [21]. VulGenie combines patch dependencies, attack/defense flows and API-rule synthesis [10]. These preclude a first-use claim for analyzer context or semantic roles. SEMWEAVER edits an existing detector and evaluates the resulting changes; its provenance interface neither implements a new feasibility procedure nor guarantees more precise detection.

10 Conclusion

SEMWEAVER exposes provenance-labelled compiler/checker context for automated refinement and makes the resulting edits and warning outcomes inspectable. Across 39 generated-only CSA subjects, native refinement recovers more vulnerable-version warning signals than KNighter’s actual loop, while nearly tying no-internal F_1 and increasing full-cohort fixed-side burden. The complete non-tie audit and selected interventions explain this divergence: recognizing a backend representation can activate a useful target report while leaving object, guard, path or scope obligations unsatisfied. Separate perturbations, CodeQL and model-subset measurements bound these findings. The study supplies concrete evidence for distinguishing checker activation, supported target diagnoses and inspection burden; it establishes neither universally better precision nor unseen-site generalization.

Acknowledgments

Original disclosure (retained verbatim): In paper writing, the authors used generative AI tools (ChatGPT and Claude) solely for language polishing and clarity-oriented editing.

For this revision, Codex also assisted with evidence synthesis and drafting; research uses are detailed in Section 5.

Data Availability

English documentation, E1–E4 records and the target-validation supplement (raw reports and offline R0/R1 checks) are at <https://anonymous.4open.science/r/SemWeaver-FD1E/>. Manual/mixed histories are unscored. Full reruns require pinned tools and model access. We intend to maintain public access after acceptance.

References

- [1] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 259–269. doi:10.1145/2594291.2594299
- [2] Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. 2016. QL: Object-Oriented Queries on Relational Data. In *Proceedings of the 30th European Conference on Object-Oriented Programming (ECOOP)*. 2:1–2:25. doi:10.4230/LIPIcs.ECOOP.2016.2
- [3] Thomas Ball and Sriram K. Rajamani. 2002. The SLAM Project: Debugging System Software via Static Analysis. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 1–3. doi:10.1145/503272.503274
- [4] Al Bessey, Ken Block, Ben Chelf, Andy Chou, Bryan Fulton, Seth Hallem, Charles Henri-Gros, Asya Kamsky, Scott McPeak, and Dawson Engler. 2010. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. *Commun. ACM* 53, 2 (2010), 66–75. doi:10.1145/1646353.1646374
- [5] Martin Bravenboer and Yannis Smaragdakis. 2009. Strictly Declarative Specification of Sophisticated Points-to Analyses. In *Proceedings of the 24th ACM SIGPLAN Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*. 243–262. doi:10.1145/1640089.1640108
- [6] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 209–224. https://www.usenix.org/legacy/event/osdi08/tech/full_papers/cadar/cadar.pdf
- [7] Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O’Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. 2015. Moving Fast with Software Verification. In *Proceedings of the 7th NASA Formal Methods Symposium (NFM)*. 3–11. doi:10.1007/978-3-319-17524-9_1
- [8] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Transactions on Software Engineering* 48, 9 (2022), 3280–3296. doi:10.1109/TSE.2021.3087402
- [9] Patrick J. Chapman, Cindy Rubio-González, and Aditya V. Thakur. 2024. Interleaving Static Analysis and LLM Prompting. In *Proceedings of the 13th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*. 9–17. doi:10.1145/3652588.3663317
- [10] Bofei Chen, Shuang Liao, Lei Zhang, Chibin Zhang, Mathias Payer, and Yuan Zhang. 2026. Patch-Guided Vulnerability Detection: Extracting Java API Security Rules via Attack–Defense Cross-Analysis. In *35th USENIX Security Symposium (USENIX Security 26)*. USENIX Association, Baltimore, MD, 1847–1866. <https://www.usenix.org/conference/usenixsecurity26/presentation/chen-bofei>
- [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374
- [12] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *Proceedings of the 12th International Conference on Computer Aided Verification (CAV)*. 154–169. doi:10.1007/10722167_15
- [13] Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. 2005. The ASTRÉE Analyzer. In *Proceedings of the 14th European Symposium on Programming (ESOP)*. 21–30. doi:10.1007/978-3-540-31987-0_3
- [14] Xueying Du, Kai Yu, Chong Wang, Yi Zou, Wentai Deng, Zuoyu Ou, Xin Peng, Lingming Zhang, and Yiling Lou. 2025. Minimizing False Positives in Static Bug Detection via LLM-Enhanced Path Feasibility Analysis. *arXiv:2506.10322* <https://arxiv.org/abs/2506.10322>
- [15] Dawson Engler, David Yu Chen, Seth Hallem, Andy Chou, and Benjamin Chelf. 2001. Bugs as Deviant Behavior: A General Approach to Inferring Errors in Systems Code. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*. 57–72. doi:10.1145/502034.502041
- [16] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*. 508–512. doi:10.1145/3379597.3387501
- [17] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A Transformer-Based Line-Level Vulnerability Prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*. 608–620. doi:10.1145/3524842.3528452
- [18] GitHub. 2026. CodeQL Documentation. <https://codeql.github.com/docs/>. Accessed: Jun. 27, 2026.

- [19] Jinyao Guo, Chengpeng Wang, Xiangzhe Xu, Zian Su, and Xiangyu Zhang. 2025. RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 21083–21100. <https://proceedings.mlr.press/v267/guo25n.html>
- [20] Sarah Heckman and Laurie Williams. 2011. A Systematic Literature Review of Actionable Alert Identification Techniques for Automated Static Code Analysis. *Information and Software Technology* 53, 4 (2011), 363–387. doi:10.1016/j.infsof.2010.12.007
- [21] Mohsen Iranmanesh, Sina Moradi Sabet, Sina Marefat, Ali Javidi Ghasr, Allison Wilson, Iman Sharafaldin, and Mohammad A. Tayebi. 2025. ZeroFalse: Improving Precision in Static Analysis with LLMs. arXiv:2510.02534 <https://arxiv.org/abs/2510.02534>
- [22] Jiyong Jang, Abeer Agrawal, and David Brumley. 2012. ReDeBug: Finding Unpatched Code Clones in Entire OS Distributions. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy*. 48–62. doi:10.1109/SP.2012.13
- [23] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why Don't Software Developers Use Static Analysis Tools to Find Bugs?. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, San Francisco, CA, 672–681. doi:10.1109/ICSE.2013.6606613
- [24] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. 2016. Soufflé: On Synthesis of Program Analyzers. In *Proceedings of the 28th International Conference on Computer Aided Verification (CAV)*. 422–430. doi:10.1007/978-3-319-41540-6_23
- [25] Nenad Jovanovic, Christopher Kruegel, and Engin Kirda. 2006. Pixy: A Static Analysis Tool for Detecting Web Application Vulnerabilities. In *Proceedings of the 2006 IEEE Symposium on Security and Privacy*. 258–263. doi:10.1109/SP.2006.29
- [26] Raphaël Khoury, Anderson R. Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. How Secure Is Code Generated by ChatGPT?. In *Proceedings of the 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2445–2451. doi:10.1109/SMC53992.2023.10394237
- [27] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 802–811. doi:10.1109/ICSE.2013.6606626
- [28] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery. In *Proceedings of the 2017 IEEE Symposium on Security and Privacy*. 595–614. doi:10.1109/SP.2017.62
- [29] Ted Kremenek and Dawson Engler. 2003. Z-Ranking: Using Statistical Analysis to Counter the Impact of Static Analysis Approximations. In *Proceedings of the 10th International Static Analysis Symposium (SAS)*. 295–315. doi:10.1007/3-540-44898-5_16
- [30] Claire Le Goues, Michael Dewey-Vogt, Stephanie Forrest, and Westley Weimer. 2012. A Systematic Study of Automated Program Repair: Fixing 55 out of 105 Bugs for \$8 Each. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*. 3–13. doi:10.1109/ICSE.2012.6227211
- [31] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2024. Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1, Article 111 (2024), 26 pages. doi:10.1145/3649828
- [32] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/582d4e27fa24168f3af1f4582655034b-Abstract-Conference.html
- [33] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Hanchao Qi, and Jie Hu. 2016. VulPecker: An Automated Vulnerability Detection System Based on Code Similarity Analysis. In *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*. 201–213. doi:10.1145/2991079.2991102
- [34] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2022. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2022), 2244–2258. doi:10.1109/TDSC.2021.3051525
- [35] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In *Proceedings of the 25th Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2018.23158
- [36] V. Benjamin Livshits and Monica S. Lam. 2005. Finding Security Vulnerabilities in Java Applications with Static Analysis. In *Proceedings of the 14th USENIX Security Symposium*. 271–286. https://www.usenix.org/legacy/events/sec05/tech/full_papers/livshits/livshits.pdf
- [37] LLVM Project. 2026. Clang Static Analyzer Documentation. <https://clang-analyzer.llvm.org/>. Accessed: Jun. 27, 2026.
- [38] Fan Long and Martin Rinard. 2015. Staged Program Repair with Condition Synthesis. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 166–178. doi:10.1145/2786805.2786811

- [39] Fan Long and Martin Rinard. 2016. Automatic Patch Generation by Learning Correct Code. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 298–312. doi:10.1145/2837614.2837617
- [40] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 46534–46594. doi:10.52202/075280-2019
- [41] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable Multiline Program Patch Synthesis via Symbolic Analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. 691–701. doi:10.1145/2884781.2884807
- [42] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. SemFix: Program Repair via Semantic Analysis. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 772–781. doi:10.1109/ICSE.2013.6606623
- [43] Yoann Padioleau, Julia L. Lawall, René Rydhof Hansen, and Gilles Muller. 2008. Documenting and Automating Colateral Evolutions in Linux Device Drivers. In *Proceedings of the 3rd ACM SIGOPS/EuroSys European Conference on Computer Systems*. 247–260. doi:10.1145/1352592.1352618
- [44] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy*. 754–768. doi:10.1109/SP46214.2022.9833571
- [45] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2785–2799. doi:10.1145/3576915.3623157
- [46] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. 2015. An Analysis of Patch Plausibility and Correctness for Generate-and-Validate Patch Generation Systems. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA)*. Association for Computing Machinery, 24–36. doi:10.1145/2771783.2771791
- [47] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. 2018. Automated Vulnerability Detection in Source Code Using Deep Representation Learning. In *Proceedings of the 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*. 757–762. doi:10.1109/ICMLA.2018.00120
- [48] Joseph R. Ruthruff, John Penix, J. David Morgenthaler, Sebastian Elbaum, and Gregg Rothermel. 2008. Predicting Accurate and Actionable Static Analysis Warnings: An Experimental Approach. In *Proceedings of the 30th International Conference on Software Engineering (ICSE)*. 341–350. doi:10.1145/1368088.1368135
- [49] Caitlin Sadowski, Jeffrey van Gogh, Ciera Jaspan, Emma Soderberg, and Collin Winter. 2015. Tricorder: Building a Program Analysis Ecosystem. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*. 598–608. doi:10.1109/ICSE.2015.76
- [50] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 68539–68551. doi:10.52202/075280-2997
- [51] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. 2012. AddressSanitizer: A Fast Address Sanity Checker. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*. USENIX Association, Boston, MA, 309–318. <https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany>
- [52] Srinivasan Shanmugam. 2024. drm/amd/display: Fix buffer overflow in get_host_router_total_dp_tunnel_bw(). Linux kernel commit 97cba232549b9fe7e491fb60a69cf93075015f29. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=97cba232549b9fe7e491fb60a69cf93075015f29> Commit metadata and source checked locally, September 27, 2026.
- [53] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 8634–8652. doi:10.52202/075280-0377
- [54] Edward K. Smith, Earl Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the Cure Worse than the Disease? Overfitting in Automated Program Repair. In *Proceedings of the 10th Joint Meeting of the European Software Engineering Conference and ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Association for Computing Machinery, 532–543. doi:10.1145/2786805.2786825
- [55] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 404–415. doi:10.1145/1168857.1168907

- [56] Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, Xiaoheng Xie, and Xiangyu Zhang. 2024. LLM DFA: Analyzing Dataflow in Code with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 131545–131574. doi:10.52202/079017-4181
- [57] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy*. 590–604. doi:10.1109/SP.2014.44
- [58] Fabian Yamaguchi, Markus Lottmann, and Konrad Rieck. 2012. Generalized Vulnerability Extrapolation Using Abstract Syntax Trees. In *Proceedings of the 28th Annual Computer Security Applications Conference (ACSAC)*. 359–368. doi:10.1145/2420950.2421003
- [59] Chenyuan Yang, Zijie Zhao, Zichen Xie, Haoyu Li, and Lingming Zhang. 2025. KNight: Transforming Static Analysis with LLM-Synthesized Checkers. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*. 655–669. doi:10.1145/3731569.3764827
- [60] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 11809–11822. doi:10.52202/075280-0517
- [61] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=WE_vluYUL-X